is out of order no matter which way you go, so a test for sorting will correctly fail until the logic is in place.

Testing Shared Modules and ActiveSupport Concerns

Often multiple models in your application share some kind of common feature set. For example, you may have multiple object types that can be purchased, tagged, or commented on. You can use standard Ruby classes and modules for this shared behavior. If the shared behavior has both class and instance methods, Rails provides ActiveSupport::Concern, which allows you to easily use a common pattern to mix multiple kinds of behavior from one module.

Testing this shared behavior can be a challenge. You don't want to have to rewrite the shared behavior specs for each class that shares the mixed-in module. At the same time, if the shared feature depends on data being available in each class, that dependency is testable logic.

RSpec has a powerful mechanism for sharing specs across multiple objects that have common functionality, simply called the *shared example*. You can use shared examples to run the same set of specs in multiple describe blocks, whether the common feature is encapsulated in a module or not.

Shared examples in RSpec have two parts: the definition and the usage. Shared examples must be defined before they're used. In a Rails context, the easiest way to do that is to put the shared examples inside a spec/shared directory and then add this line to the Rails rails_helper.rb file to load everything in that directory before any specs are run:

```
Dir[Rails.root.join("spec/shared/**/*.rb")].each { |f| require f }
```

Let's create a contrived example, suggesting we want projects and tasks to respond to a similar set of adjectives about their size. Here a shared group is created, with the method shared_examples taking a string argument and a block:

model/01/spec/shared/size_group.rb

```
RSpec.shared_examples "sizeable" do
  let(:instance) { described_class.new }

  it "knows a one-point story is small" do
    allow(instance).to receive(:size).and_return(1)
    expect(instance).to be_small
  end

  it "knows a five-point story is epic" do
    allow(instance).to receive(:size).and_return(5)
    expect(instance).to be_epic
  end
end
```